MOHAMED BIN ZAYED UNIVERSITY OF ARTIFICIAL INTELLIGENCE

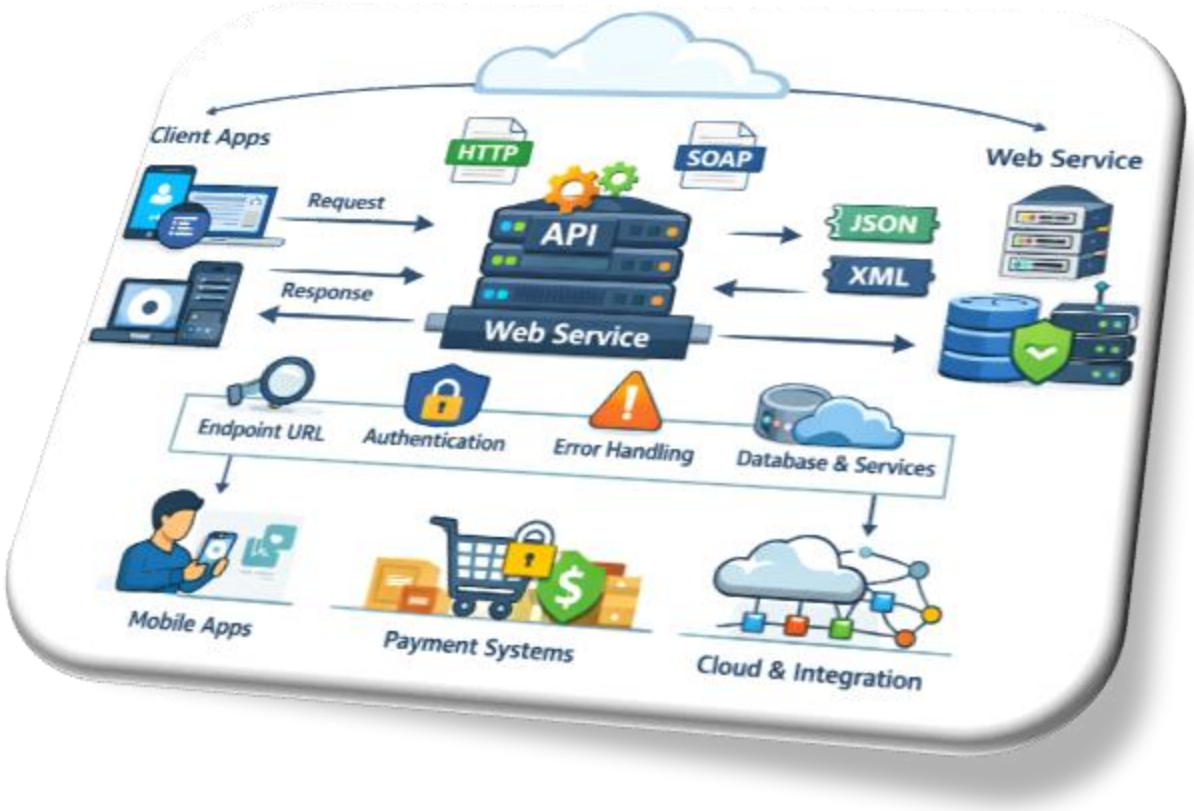
Mohamed bin Zayed University of Artificial Intelligence

SWM Engineering

Date
5/2/2026

4.2

Web Services



What is a Web Service?

A **web service** is a software component that:

- Is accessible over a network
- Exposes functionality via a well-defined interface
- Communicates using standard protocols and data formats
- Enables **machine-to-machine** interaction

Key Characteristics:

- **Interoperability** – Works across different platforms and languages
- **Loose coupling** – Client and service evolve independently
- **Standardized communication** – Uses HTTP, XML, JSON, etc.
- **Discoverable** – Interface and usage are clearly defined
- **Statelessness (often)** – Each request is independent

Service-Oriented Architecture (SOA) is an architectural style that organizes software functionality as reusable, loosely coupled services communicating over a network.

Commonly Used Data Formats

The most common data formats used in data exchange are:

- **XML** (eXtensible Markup Language): a **text-based format** used to **store and exchange structured data** in a readable and platform-independent way.

- **JSON** (JavaScript Object Notation): a **lightweight, text-based data format** used to **store and exchange data** between applications.

XML and JSON

XML

```xml
<library>
  <book>
    <id>1</id>
    <title>Clean Code</title>
    <author>Robert Martin</author>
    <available>true</available>
  </book>

  <member>
    <id>101</id>
    <name>Alice</name>
  </member>
</library>
```

JSON

```json
{
  "library": {
    "book": {
      "id": 1,
      "title": "Clean Code",
      "author": "Robert Martin",
      "available": true
    },
    "member": {
      "id": 101,
      "name": "Alice"
    }
  }
}
```

XML Usage

Area	Use of XML	Example
Web Services	Message format in SOAP	SOAP request/response
Data Exchange	Sharing data between systems	B2B integrations
Configuration Files	Application configuration	`web.xml`, `pom.xml`
Document Storage	Structured document representation	DOCX, XLSX
Data Validation	Enforcing data structure	XSD, DTD
Enterprise Messaging	Standard message format	ESB systems
Metadata	Describing data	Publishing, scientific data
System Integration	Linking heterogeneous systems	Banking systems

- XSD (XML Schema Definition) is used to define and validate the structure of an XML document
- DTD (Document Type Definition) is used to define the structure and rules of an XML document
- ESB (Enterprise Service Bus) is a middleware architecture used to connect and integrate multiple applications and services within an enterprise.

JSON Usage

Area	Use of JSON	Example
REST Web Services	Request and response data format	REST APIs
Web Applications	Data exchange between UI and server	AJAX / Fetch
Mobile Applications	API communication	Android / iOS apps
Configuration Files	Application settings	`package.json`
Data Storage	Lightweight data storage	NoSQL (MongoDB)
Data Serialization	Object-to-text conversion	Client–server transfer
Microservices	Service-to-service communication	REST / HTTP
APIs & Integration	Third-party integrations	Payment APIs

AJAX (Asynchronous JavaScript and XML) is a technique used in web applications to communicate with a server without reloading the web page.

XML vs JSON

Advantages (Pros) of XML

- Platform independent and language independent
- Self-descriptive (tags describe the data)
- Supports hierarchical and structured data
- Strong validation support using **DTD** and **XSD**
- Widely supported by tools and systems
- W3C standardized format

Disadvantages (Cons) of XML

- Very verbose (many opening and closing tags)
- Larger file size compared to JSON
- Slower parsing and processing
- Complex syntax for beginners
- Not suitable for lightweight web and mobile applications

XML vs JSON

Advantages (Pros) of JSON

- Lightweight and compact data format
- Easy to read and write
- Faster parsing and processing than XML
- Naturally supported by JavaScript
- Widely used in REST web services
- Reduces network bandwidth usage

Disadvantages (Cons) of JSON

- No strong built-in schema validation (basic level)
- Less strict structure compared to XML
- Not suitable for complex document structures
- Limited support for metadata
- Comments are not allowed

XML and JSON

Feature	XML	JSON
Full form	eXtensible Markup Language	JavaScript Object Notation
Type	Markup language	Data interchange format
Syntax style	Tag-based	Key–value pairs
Readability	Verbose	Compact
Data size	Larger	Smaller
Parsing speed	Slower	Faster
Schema support	Strong	Limited (JSON Schema – optional)
Data types	Text only	Multiple data types
Case sensitive	Yes	Yes
Comments	Allowed	Not allowed

A Note on Schemaless

Question: when a schemaless format is useful?

CSV

```csv
id,name,email,age
1,Alice,alice@example.com,21
2,Bob,,22
3,Charlie,charlie@example.com,
```

JSON

```json
[
  {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "age": 21
  },
  {
    "id": 2,
    "name": "Bob",
    "age": 22
  },
  {
    "id": 3,
    "name": "Charlie",
    "email": "charlie@example.com"
  }
]
```

Types of Web Services

Type	Description	Data Format	Key Features	Use Cases
REST	Resource-based web service	JSON	Lightweight, scalable, stateless	Web apps, mobile apps
SOAP	Protocol-based web service	XML	Secure, reliable, strict standards	Banking, enterprise systems
GraphQL	Query-based API	JSON	Flexible queries, single endpoint	Modern frontends
RPC	Remote procedure calls	JSON / XML	Function-oriented	Internal services

- REST (Representational State Transfer)
- SOAP (Simple Object Access Protocol)
- GraphQL (Graph Query Language)
- RPC (Remote Procedure Call)

REST

REST (Representational State Transfer)

- Architectural style for building web services
- Introduced by **Roy Fielding** (2000)
- Uses HTTP and standard web principles
- Most widely used API style today
- Operates on resources identified by URLs
- Emphasizes simplicity and scalability
- Language agnostic

REST

HTTP Method	Purpose	REST Meaning
GET	Retrieve data	Read a resource
POST	Send data	Create a new resource
PUT	Send data	Update entire resource
PATCH	Send data	Update part of resource
DELETE	Remove data	Delete a resource

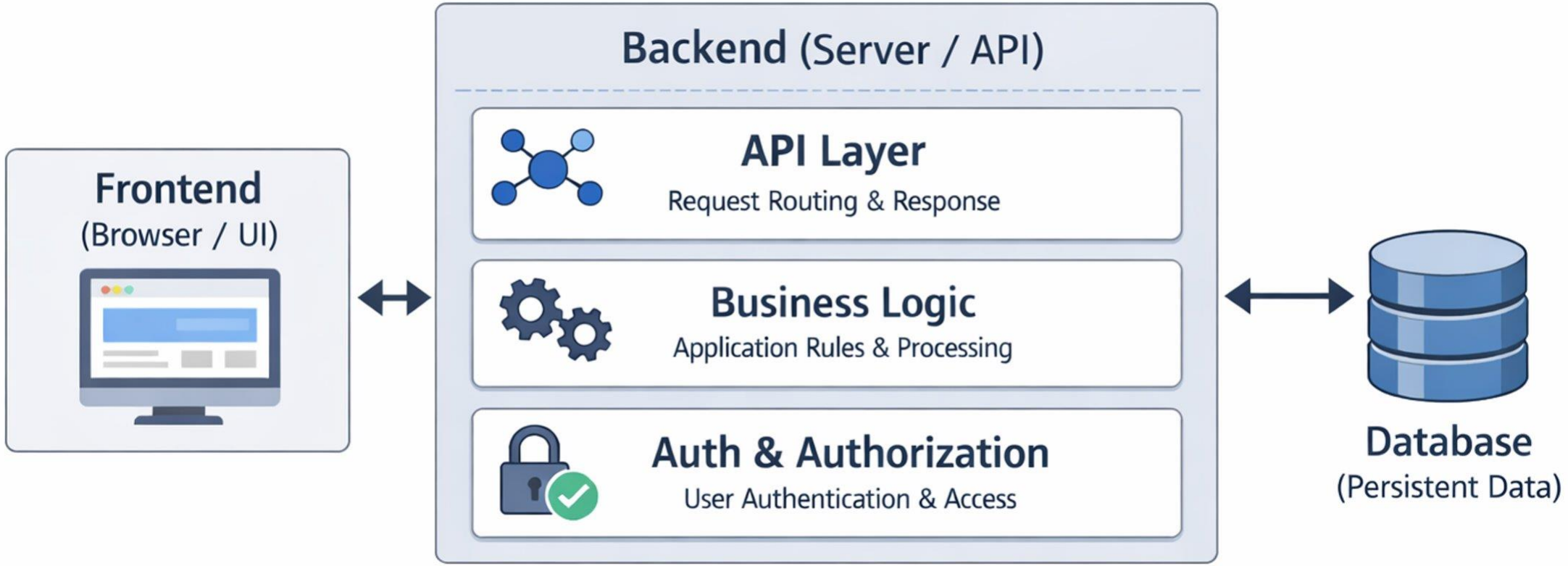
What Can Be a Resource?

A resource can represent:

- An object (user, product, order)
- A collection (users, products)
- A relationship (user's orders)
- A process result (report, search result)

The Application

We will design a **Library System** using **HTML + JavaScript (frontend)**, **Node.js (backend)**, and **SQL Server (database)**.



A Simple Architecture

HTML + JavaScript (browser) [**frontend**]
|
fetch()
|
Node.js + Express (REST API) [**backend**]
|
SQL queries
|
SQL Server [**database**]

Node.js + Express

Node.js is a free, open-source **JavaScript runtime** that runs on Windows, Mac, Linux, and more. It lets you execute JavaScript code outside of a web browser, enabling server-side development with JavaScript. Built on Chrome's V8 JavaScript engine, **Node.js is designed for building scalable network applications efficiently.**

We will use Node.js for the backend together with Express.

Node.js excels at handling many simultaneous connections with minimal overhead, making it perfect for:

- Real-time applications (chats, gaming, collaboration tools)
- APIs and microservices
- Data streaming applications
- Command-line tools
- Server-side web applications

Express provides a structure, a set of tools (like routing and middleware), and guidelines that simplify the process of building server-side applications and APIs. It offers a higher level of abstraction than using raw Node.js HTTP modules, handling common tasks so developers can focus on application logic.

Resources of the Library System

Resource A: Books

- Collection: `/api/books`
- Single book: `/api/books/:id`

Resource B: Loans

- Collection: `/api/loans`
- Single loan: `/api/loans/:id`

The Database

```
CREATE TABLE Books (
  BookId INT IDENTITY(1,1) PRIMARY KEY,
  Title NVARCHAR(200) NOT NULL,
  Author NVARCHAR(120) NOT NULL,
  IsAvailable BIT NOT NULL DEFAULT 1
);

CREATE TABLE Loans (
  LoanId INT IDENTITY(1,1) PRIMARY KEY,
  BookId INT NOT NULL,
  BorrowerName NVARCHAR(120) NOT NULL,
  BorrowedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
  ReturnedAt DATETIME2 NULL,

  CONSTRAINT FK_Loans_Books FOREIGN KEY (BookId) REFERENCES Books(BookId)
);
```

Simple Rules

- If `Books.IsAvailable = 1` → can borrow
- When borrowed → set `IsAvailable = 0`, create a row in `Loans`
- When returned → set `IsAvailable = 1`, set `ReturnedAt` in `Loans`

REST Endpoints

Method	Endpoint	What it does
GET	`/api/books`	list all books
POST	`/api/books`	add a new book

Method	Endpoint	What it does
POST	`/api/loans`	borrow a book
PATCH	`/api/loans/{loanid}`	return a book

Queries

Get all books

GET `/api/books`

Response

```
[
  { "bookId": 1, "title": "Clean Code", "author": "Robert Martin", "isAvailable": true },
  { "bookId": 2, "title": "REST Basics", "author": "Fielding", "isAvailable": false }
]
```

Add a book

POST `/api/books`

```
{ "title": "Database Systems", "author": "Elmasri" }
```

Response

```
{ "bookId": 3, "title": "Database Systems", "author": "Elmasri", "isAvailable": true }
```

Queries

Borrow a book

POST `/api/loans`
{ "bookId": 3, "borrowerName": "Ali" }

Response

{ "loanId": 10, "bookId": 3, "borrowerName": "Ali", "borrowedAt": "2026-02-04T10:00:00Z" }

If the book is not available:
{ "error": "Book is already borrowed" }

Return a book

PATCH `/api/loans/10/return`

Response
{ "loanId": 10, "returnedAt": "2026-02-04T12:00:00Z" }

How to Trigger these Queries in the Frontend?

index.html

```html
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Library UI</title>
</head>

<body>
  <h1>Library System</h1>

  <h2>All Books</h2>
  <button id="btnLoadBooks">Load Books</button>

  <h2>Add Book</h2>
  <input id="bookTitle" placeholder="Title" />
  <input id="bookAuthor" placeholder="Author" />
  <button id="btnAddBook">Add</button>
```

```html
<h2>Borrow Book</h2>
  <input id="borrowBookId"
placeholder="BookId" />
  <input id="borrowerName"
placeholder="Borrower name" />
  <button id="btnBorrow">Borrow</button>

  <h2>Return Book</h2>
  <input id="returnLoanId"
placeholder="LoanId" />
  <button id="btnReturn">Return</button>

  <h2>Output</h2>
  <pre id="output"></pre>

  <script src="app.js"></script>
</body>
</html>
```

How to Trigger these Queries in the Frontend?

app.js

```
const output = document.getElementById("output");
// Helper: show any JS object in the function
show(data) {
  output.textContent = typeof data === "string" ?
data : JSON.stringify(data, null, 2);
}

// Helper: safe JSON parsing (some responses like
204 may have no body)
async function safeJson(res) {
  const text = await res.text();
  return text ? JSON.parse(text) : null;
}
```

```
/* 1) GET /api/books → Load all books */

async function loadBooks() {
 try {
  const res = await fetch("/api/books");
  if (!res.ok) {
   const err = await safeJson(res);
   return show(err || { error: "Failed to
load books" });   }
  const books = await res.json();
  show(books);
 } catch (e) {
   show({ error: e.message });
 }
}
```

How to Trigger these Queries in the Frontend?

app.js

```
const output = document.getElementById("output");
// Helper: show any JS object in the function
show(data) {
  output.textContent = typeof data === "string" ?
data : JSON.stringify(data, null, 2);
}

// Helper: safe JSON parsing (some responses like
204 may have no body)
async function safeJson(res) {
  const text = await res.text();
  return text ? JSON.parse(text) : null;
}
```

```
/* 1) GET /api/books → Load all books */

async function loadBooks() {
 try {
  const res = await fetch("/api/books");
  if (!res.ok) {
   const err = await safeJson(res);
   return show(err || { error: "Failed to
load books" });   }
  const books = await res.json();
  show(books);
 } catch (e) {
   show({ error: e.message });
 }
}
```

Mohamed bin Zayed
University of
Artificial Intelligence

Thank you